Event Stream Reference

Cogniflow Orchestrator v1

Event stream reference

Every time you run a pipeline, the orchestrator writes a diary of what happened, one line at a time, into a file called `events.jsonl` inside the pipeline's `.state/` folder. Each line is a single "event" — something worth recording, like "agent started", "agent finished", "this layer took 25 seconds".

This document walks through those events **in the exact order they appear in the stream**, using real data from the successful run `20260419-155654` of the `10-software-factory` pipeline.

How to read this document

For every event you will find two sections:

- **What it means** — a plain-English explanation anyone can follow.
- **TECH** — the implementation details (which Python function emits it, which files are touched, how numbers are computed). Skip this section if you are not a developer.

A quick mental model

Think of the orchestrator as a factory manager:

- A **pipeline** is the whole production run.
- A pipeline is split into **layers**. Each layer is a group of workers (agents) that can run at the same time.
- Each **agent** is one worker. It reads some input files, asks Claude to produce something, and writes the result to an output file.

The event stream is the manager's notebook — one line every time something changes.

Event shape

Every line in `events.jsonl` is a small JSON object:

```
{"ts": "2026-04-19T15:56:54Z", "event": "pipeline_start", "name": "10-software-factory",
         "run_id": "20260419-155654", "total_agents": 5}
```

- `ts` — the time the event was recorded (UTC).
- `event` — the name of the event (e.g. `agent_done`).
- The rest of the fields depend on the event type.

The stream, in order

The table below lists the events **in the order they typically appear** for a run with no failures and no routers.

#	Event	When it fires
1	`pipeline_start`	Once, at the very beginning
2	`layer_start`	Start of each layer
3	`agent_launched`	When an agent begins its preparation
4	`agent_inputs_collected`	After input files are copied into the agent's folder
5	`agent_budget_estimated`	After a pre-flight size check
6	`agent_context_ready`	After the final prompt file is built on disk
7	`agent_start`	Right before Claude is called
8	`agent_done`	When Claude returns a valid answer
9	`layer_done`	When every agent in the layer has finished
—	(steps 2–9 repeat for every layer)	
10	`pipeline_done`	Once, at the very end

Events 3–8 repeat for every agent inside a layer. If two agents in a layer run in parallel, their events are **interleaved** — don't assume each agent's block is contiguous.

There are a few extra events that only fire when something unusual happens (a failure, a timeout, a skipped agent, a routing decision). Those are covered at the end.

---

1. `pipeline_start`

What it means

The pipeline has officially begun. The orchestrator has already read the pipeline's configuration file, confirmed everything looks sane, and is about to start the first layer.

Example from the real run:

```
{"ts": "2026-04-19T15:56:54Z", "event": "pipeline_start",
 "name": "10-software-factory", "run_id": "20260419-155654",
 "total_agents": 5}
```

You now know: the pipeline is `10-software-factory`, this particular run has a unique ID (`20260419-155654`, built from the start time), and there are 5 agents scheduled.

> **TECH**
>
> Emitted by `core.py:91` inside `run_pipeline()`, after the DAG has been validated and `len(graph.nodes)` has been stored as `total`. No files are read at emit time — the fields come from `pipeline.json`, which the orchestrator parsed seconds earlier. The `run_id` format is `YYYYMMDD-HHMMSS` in UTC.

---

2. `layer_start`

What it means

A layer is a batch of agents that are allowed to run at the same time (because none of them depend on each other). This event announces which agents are in the batch and whether they will really run in parallel or one after another.

Example from the real run:

```
{"ts": "2026-04-19T15:58:32Z", "event": "layer_start",
 "layer": 3, "agents": ["004_tester", "005_documenter"],
 "parallel": true}
```

Layer 3 contains the tester and the documenter, and they will run side by side.

> **TECH**
>
> Emitted by `core.py:124` at the top of the per-layer loop. The list of agents in each layer is produced by `dag.py` using `networkx.topological_generations()` — a standard way of flattening a DAG into "you can run these together" groups. `parallel: true` means the orchestrator will spawn them in a `ThreadPoolExecutor`.

---

3. `agent_launched`

What it means

The orchestrator has picked up one specific agent and is about to prepare it. “Prepare” means: gather its input files, check they are not too big, and write the final prompt that Claude will receive.

Example:

```
{"ts": "2026-04-19T15:56:54Z", "event": "agent_launched",
 "agent": "001_spec"}
```

> **TECH**
>
> Emitted by `agent.py:104` inside `exec_agent()`, **after** the resume guard (which checks `06_status.json` to see if the agent was already done in a previous run). If the agent is already done, you see `agent_skip` instead and no further agent-level events are emitted for it.

---

4. `agent_inputs_collected`

What it means

The agent’s input folder has just been filled in. An agent can receive two kinds of inputs:

- **Upstream outputs** — whatever previous agents produced, because this agent depends on them. Example: the architect needs the spec that the spec agent wrote.
- **Static inputs** — shared files that live next to the pipeline configuration and that the agent was told to read (e.g. a fixture file, a reference document).

`input_count` is the total number of files copied in.

Example from the real run:

Agent	input_count	What was copied
`001_spec`	0	nothing — it’s the first agent
`002_architect`	1	the spec produced by `001_spec`
`003_developer`	1	the design produced by `002_architect`
`004_tester`	1	the code produced by `003_developer`
`005_documenter`	1	the code produced by `003_developer`

```
{"ts": "2026-04-19T15:57:15Z", "event": "agent_inputs_collected",
 "agent": "002_architect", "input_count": 1}
```

TECH

Emitted by `context.py:134` at the end of `collect_inputs()`. For every dependency the function copies the upstream agent's `05_output.<run_id>.md` (or its `05_output.md` symlink as a fallback) into `agents/<this_agent>/03_inputs/from_<dep_id>.md`. For every path listed in `00_config.json → static_inputs` it copies the file to `03_inputs/static/<filename>`. `input_count = upstream_count + static_count`.

---

5. `agent_budget_estimated`

What it means

Before actually building the final prompt, the orchestrator does a quick **pre-flight size check**: how big, roughly, will the full request to Claude be? It adds up the size of three things:

1. The **system prompt** (`01_system.md`) — the agent's role.
2. The **task prompt** (`02_prompt.md`) — what it must do.
3. All the **input files** collected in the previous step.

It then converts the total character count into an approximate token count (1 token ≈ 3.5 characters) and logs the result. If this number is too big, the orchestrator will apply a strategy to shrink it (drop the smallest inputs, summarise them, or fail loudly). If the number is fine, nothing further happens and the pipeline moves on.

This is essentially a smoke alarm: it fires every time so you can see the numbers, but it only **triggers an action** when things are actually too big.

Example from the real run:

```
{"ts": "2026-04-19T15:56:54Z", "event": "agent_budget_estimated",
 "agent": "001_spec", "context_bytes": 2758, "tokens_estimated": 788}
```

TECH

Emitted by `budget.py:60` inside `check_and_prepare_inputs()`, which runs **before** `assemble_context()`. Computation:

```
CHARS_PER_TOKEN = 3.5
total  = int(len(system_md) / 3.5)
total += int(len(prompt_md) / 3.5)
for f in sorted(inputs_dir.glob("*.md")):
    total += int(len(f.read_text()) / 3.5)
log.agent_budget_estimated(agent_id,
                           bytes_=int(total * 3.5),
                           tokens_est=total)
```

Note that `context_bytes` here is **not** a real byte count of any file — it is the token estimate multiplied back by 3.5. If `tokens_estimated > orchestrator.input_token_budget`, an `agent_budget_exceeded` event follows and the configured strategy runs.

---

6. `agent_context_ready`

What it means

The orchestrator has now built the **real prompt file** (`04_context.md`) that Claude will read as its user message. This event reports its actual size on disk.

`04_context.md` is the authoritative record of what Claude was asked. It always starts with the task prompt. If there are static inputs, they come next. If there are upstream outputs, they come last.

Important: `04_context.md` does **not** include the system prompt (`01_system.md`). The system prompt is passed to Claude through a separate channel (the `--system-prompt` command-line flag), not as part of the user message.

That is why this event usually reports a **smaller** number than `agent_budget_estimated` — the size of the system prompt is missing.

Example from the real run (same agent as the previous section):

```
{"ts": "2026-04-19T15:56:54Z", "event": "agent_context_ready",
 "agent": "001_spec", "context_bytes": 1846, "tokens_estimated": 527}
```

The two “size” events side by side

For every agent in the run, the two events agree on the shape of the request but differ by the weight of `01_system.md`:

Agent	budget_estimated bytes	context_ready bytes	Δ (≈ system prompt)
`001_spec`	2,758	1,846	912
`002_architect`	5,299	4,471	828
`003_developer`	7,084	6,055	1,029
`004_tester`	2,880	1,855	1,025
`005_documenter`	2,940	1,936	1,004

How `04_context.md` is put together (concrete example)

Take `002_architect`. Before this event fires, the agent folder contains:

- `02_prompt.md` (1,952 bytes) — the task the architect must perform
- `03_inputs/from_001_spec.md` (2,448 bytes) — the spec produced by `001_spec`
- no `03_inputs/static/` folder (no static inputs were declared)

The orchestrator assembles `04_context.md` in this order:

1. Start with the task prompt, under a heading `# Task`.
2. If there are static inputs, add a section `# Input files` with each file as a sub-section `## <filename>` wrapped in a fenced code block. (Skipped here — no static inputs.)
3. If there are upstream outputs, add a section `# Context from upstream agents` with each file as a sub-section `## Output from: <agent_id>`.

Separators `---` are inserted between sections. The resulting file looks like:

```
# Task

<contents of 02_prompt.md>

---

# Context from upstream agents

## Output from: 001_spec

<contents of from_001_spec.md>
```

Final size on disk: 4,554 bytes. The event logs 4,471 — the small gap is due to trailing whitespace trimmed from each section.

TECH

Emitted by `context.py:190` at the end of `assemble_context()`, right after `04_context.md` is written. Computation:

```
full_context = "\n\n---\n\n".join(parts)
context_path.write_text(full_context, encoding="utf-8")
ctx_bytes  = len(full_context.encode("utf-8"))
ctx_tokens = int(ctx_bytes / 3.5)
log.agent_context_ready(agent_id, ctx_bytes, ctx_tokens)
```

`context_bytes` is the UTF-8 byte length of the in-memory string (which is identical to the bytes just written, modulo any trailing newline added by the OS). Because the system prompt is excluded, this number is always smaller than the corresponding `agent_budget_estimated`.

At Claude-invocation time (`agent.py:170`), the subprocess is called as:

```
subprocess.run(
    [claude_bin, "--system-prompt", <01_system.md>, "-p", ...flags],
    input=<04_context.md bytes>,   # piped to stdin
    ...
)
```

Nothing else is passed to Claude.

---

7. `agent_start`

What it means

Claude has been called. The clock is now running on this agent's subprocess. Everything before this point was preparation; `agent_start` marks the beginning of the actual LLM work.

Example:

```
{"ts": "2026-04-19T15:56:54Z", "event": "agent_start",
 "agent": "001_spec"}
```

TECH

Emitted by `agent.py:127` immediately after `06_status.json` is updated to `status: running` and a `time.monotonic()` stamp is taken, then right before `subprocess.run([claude_bin, ...])`.

---

8. `agent_done`

What it means

Claude returned an answer, the orchestrator saved it to disk, and everything looks healthy. This event reports:

- `duration_s` — how long Claude took, in seconds.
- `output_bytes` — the size of the file Claude wrote.
- `run_id` — the pipeline run this output belongs to.

Example from the real run:

```
{"ts": "2026-04-19T15:57:15Z", "event": "agent_done",
 "agent": "001_spec", "duration_s": 20.4, "output_bytes": 2448,
 "run_id": "20260419-155654"}
```

Agent durations varied significantly in this run: `005_documenter` finished in 55 seconds while `004_tester` (running in parallel) took over 4 minutes.

> **TECH**
>
> Emitted by `agent.py:237` after these conditions are met: exit code 0, output file exists, output file size > 0. Just before the emit, the symlink `05_output.md` is repointed to the versioned file `05_output.<run_id>.md` (on Windows without developer mode, a copy is made instead). The duration is `time.monotonic() - t0`.

---

9. `layer_done`

What it means

Every agent in the current layer has finished successfully. The pipeline is about to move on to the next layer.

Example:

```
{"ts": "2026-04-19T16:02:37Z", "event": "layer_done",
 "layer": 3, "duration_s": 245.4}
```

A layer's duration is the wall-clock time between `layer_start` and `layer_done` — i.e. the slowest agent in the layer. In this run `004_tester` dominated layer 3.

TECH

Emitted by `core.py:140`. The loop then advances to the next topological generation, or falls through to `pipeline_done` if there are no more layers.

---

10. `pipeline_done`

What it means

The whole run is over and successful. The orchestrator reports how many layers were executed and the total wall-clock time.

Example from the real run:

```
{"ts": "2026-04-19T16:02:37Z", "event": "pipeline_done",
 "run_id": "20260419-155654", "layers": 4, "duration_s": 343.1}
```

5 agents, 4 layers, 5 minutes 43 seconds in total.

TECH

Emitted by `core.py:144` once every layer has exited cleanly. If any layer raised, you see `pipeline_error` instead and `pipeline_done` is skipped entirely.

Unusual events

These events only appear when something out of the ordinary happens. They are absent from the example run, which finished cleanly.

`agent_skip`

What it means. The agent's output already exists from a previous run, so the orchestrator skipped it to save time.

TECH. Emitted by `agent.py:100` when `06_status.json` reports `status: done`. Governed by the resume guard `is_done()`.

`agent_fail`

What it means. Claude returned an error (or produced an empty output file). The pipeline stops at the end of the current layer.

TECH. Emitted by `agent.py:208` when exit code is non-zero or the versioned output file is missing/empty. Fields: `exit_code`, `duration_s`, `reason` (first 300 chars of stderr).

`agent_timeout`

What it means. Claude took too long to answer and was killed.

TECH. Emitted by `agent.py:185` on `subprocess.TimeoutExpired`. The timeout value comes from `orchestrator.agent_timeout` in `pipeline.json`.

`agent_budget_exceeded`

What it means. The pre-flight size check found that the request would be too big. The orchestrator is about to apply the agent's `budget_strategy` — drop small inputs, auto-summarise them, or fail.

TECH. Emitted by `budget.py:69`, immediately after `agent_budget_estimated`, only when `tokens_estimated > budget`. Fields: `tokens_estimated`, `budget`, `strategy` (one of `hard_fail`, `select_top_n`, `auto_summarise`).

`layer_fail`

What it means. At least one agent in the layer failed. The pipeline will stop.

TECH. Emitted by `core.py:134` inside the per-layer `except` handler, followed immediately by a `pipeline_error`.

`pipeline_error`

What it means. The run stopped with an error. `reason` explains what category of problem (validation, layer failure, etc.).

TECH. Emitted by `core.py:84` (validation failure before any layer runs) or `core.py:135` (a layer failed). Fields: `reason`, `detail`.

`router_decision` / `agent_bypassed`

What it means. Some agents act as routers: after they finish, they inspect the situation and decide which downstream branch the pipeline should follow. `router_decision` records that choice. Agents on the branches that were **not** chosen get an `agent_bypassed` event each, so the event stream still accounts for every agent in the DAG.

TECH. Both emitted by `agent.py:_evaluate_router()` after the router agent's own `agent_done`. Triggered when the agent's `00_config.json` contains a `router` block and the agent writes a `routing.json` file. Not present in the `10-software-factory` pipeline.

Quick queries

Once you understand the events, the stream is easy to slice. Run these from inside a pipeline directory.

Compare the two size events side by side:

```
python -c "
import json
est, ready = {}, {}
for l in open('.state/events.jsonl'):
    e = json.loads(l)
    if e['event'] == 'agent_budget_estimated': est[e['agent']]  = e['tokens_estimated']
    if e['event'] == 'agent_context_ready':    ready[e['agent']] = e['tokens_estimated']
for a in est:
    print(f'{a:20s} budget={est[a]:5d}  context={ready[a]:5d}  Δ={est[a]-ready[a]:+d}')
"
```

Per-agent wall-clock time and output size:

```
python -c "
import json
for l in open('.state/events.jsonl'):
    e = json.loads(l)
    if e['event'] == 'agent_done':
        print(f\"{e['agent']:20s} {e['duration_s']:6.1f}s  {e['output_bytes']:>7} bytes\")
"
```

Everything that went wrong:

```
python -c "
import json
for l in open('.state/events.jsonl'):
    e = json.loads(l)
    if e['event'] in ('agent_fail','agent_timeout','layer_fail','pipeline_error'):
        print(e)
"
```